assignment-06-code

November 4, 2026

1 Assignment 06

1.1 Import External Libraries

[1]:
```
import numpy as np
import sympy as sp

from IPython.display import display, Math
```

1.2 1. Power of Quantum Register

[2]:
```
# --- Problem 1: how many qubits fit in the world's memory? ---
WORLD_BYTES = 1e21    # the whole world has less than 10^21 bytes of memory
BYTES_PER_COEFF = 2   # one byte for the real part, one for the imaginary part

# an n-qubit register has 2^n coefficients  ->  2 * 2^n = 2^(n+1) bytes
n_max = int(np.floor(np.log2(WORLD_BYTES / BYTES_PER_COEFF)))

display(Math(r"n_{\max} = \left\lfloor \log_2 \frac{10^{21}}{2} \right\rfloor = "
             + str(n_max)))

for n in (n_max, n_max + 1):
    needed = BYTES_PER_COEFF * 2.0 ** n
    verdict = "fits" if needed <= WORLD_BYTES else "too big"
    print(f"n = {n} qubits  ->  {needed:.3e} bytes  ({verdict})")
```

$$n_{\max} = \left\lfloor \log_2 \frac{10^{21}}{2} \right\rfloor = 68$$

```
n = 68 qubits  ->  5.903e+20 bytes  (fits)
n = 69 qubits  ->  1.181e+21 bytes  (too big)
```

1.3 5. Entanglement Entropy of a Product State

[3]:
```
# --- Problem 5: density matrix and entanglement entropy of a product state ---
ket0, ket1 = np.array([1, 0]), np.array([0, 1])


def partial_trace_B(rho):
    """Tr_B(rho) for a 2-qubit rho: contract the B index (rows/cols are (a,b))."""
    return np.einsum("abcb->ac", rho.reshape(2, 2, 2, 2))


def entanglement_entropy(rho_A, tol=1e-12):
    """S = -Tr(rho log2 rho), taken on the eigenvalues, with 0 log 0 := 0."""
    eigvals = np.linalg.eigvalsh(rho_A).real
```

```
    eigvals = eigvals[eigvals > tol]
    return float(-np.sum(eigvals * np.log2(eigvals))) + 0.0


psi = np.ones(4) / 2   # (|00> + |01> + |10> + |11>) / 2
rho = np.outer(psi, psi.conj())
display(Math(r"\rho = \ket{\psi}\bra{\psi} = " + sp.latex(sp.Matrix(np.round(rho, 4)))))

# it is a product state:  |psi> = |+> (x) |+>
plus = np.array([1, 1]) / np.sqrt(2)
print("Problem 5  --  |psi> == |+> (x) |+> ?", np.allclose(psi, np.kron(plus, plus)))

print("---------------")

rho_A = partial_trace_B(rho)
display(Math(r"\rho_A = \mathrm{Tr}_B(\rho) = " + sp.latex(sp.Matrix(np.round(rho_A, 4)))))
print("Problem 5  --  eigenvalues of rho_A:", np.round(np.linalg.eigvalsh(rho_A).real, 4))
print("Problem 5  --  S = -Tr(rho_A log2 rho_A) =", entanglement_entropy(rho_A))
```

$$\rho = |\psi\rangle\langle\psi| = \begin{bmatrix} 0.25 & 0.25 & 0.25 & 0.25 \\ 0.25 & 0.25 & 0.25 & 0.25 \\ 0.25 & 0.25 & 0.25 & 0.25 \\ 0.25 & 0.25 & 0.25 & 0.25 \end{bmatrix}$$

```
Problem 5  --  |psi> == |+> (x) |+> ? True
---------------
```

$$\rho_A = \mathrm{Tr}_B(\rho) = \begin{bmatrix} 0.5 & 0.5 \\ 0.5 & 0.5 \end{bmatrix}$$

```
Problem 5  --  eigenvalues of rho_A: [0. 1.]
Problem 5  --  S = -Tr(rho_A log2 rho_A) = 0.0
```

1.4 6. Partial Tracing

[4]:
```
# --- Problem 6: entanglement entropy of |phi-> by tracing out the B space ---
phi_minus = (np.kron(ket0, ket0) - np.kron(ket1, ket1)) / np.sqrt(2)
rho = np.outer(phi_minus, phi_minus.conj())
display(Math(r"\rho = \ket{\phi^-}\bra{\phi^-} = " + sp.latex(sp.Matrix(np.round(rho, 4)))))

rho_A = partial_trace_B(rho)
display(Math(r"\rho_A = \mathrm{Tr}_B(\rho) = " + sp.latex(sp.Matrix(np.round(rho_A, 4)))))
print("Problem 6  --  eigenvalues of rho_A:", np.round(np.linalg.eigvalsh(rho_A).real, 4))
print("Problem 6  --  S_B = -Tr_A(rho_A log2 rho_A) =", entanglement_entropy(rho_A))

print("---------------")

# tracing the other space instead must give the same number (Schmidt decomposition)
rho_B = np.einsum("abad->bd", rho.reshape(2, 2, 2, 2))
print("Problem 6  --  S_A = -Tr_B(rho_B log2 rho_B) =", entanglement_entropy(rho_B))
```

$$\rho = |\phi^-\rangle\langle\phi^-| = \begin{bmatrix} 0.5 & 0.0 & 0.0 & -0.5 \\ 0.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.0 \\ -0.5 & 0.0 & 0.0 & 0.5 \end{bmatrix}$$

$$\rho_A = \mathrm{Tr}_B(\rho) = \begin{bmatrix} 0.5 & 0.0 \\ 0.0 & 0.5 \end{bmatrix}$$

```
Problem 6  --  eigenvalues of rho_A: [0.5 0.5]
Problem 6  --  S_B = -Tr_A(rho_A log2 rho_A) = 1.0
```

```
---------------
Problem 6  --  S_A = -Tr_B(rho_B log2 rho_B) = 1.0
```

1.5 7. Unitarity of the NOT Gate

[5]:
```
# --- Problem 7: the 2-qubit NOT gate X (x) X is unitary ---
X = np.array([[0, 1], [1, 0]])
NOT2 = np.kron(X, X)

display(Math(r"X \otimes X = " + sp.latex(sp.Matrix(NOT2))))
display(Math(r"(X \otimes X)^\dagger (X \otimes X) = "
             + sp.latex(sp.Matrix(NOT2.conj().T @ NOT2))))

print("Problem 7  --  unitary?",
      np.allclose(NOT2.conj().T @ NOT2, np.eye(4)),
      np.allclose(NOT2 @ NOT2.conj().T, np.eye(4)))
```

$$X \otimes X = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

$$(X \otimes X)^\dagger (X \otimes X) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

```
Problem 7  --  unitary? True True
```

1.6 8. Unitarity of the CNOT Gate

[6]:
```
# --- Problem 8: the CNOT gate is unitary ---
CNOT = np.array([[1, 0, 0, 0],
                 [0, 1, 0, 0],
                 [0, 0, 0, 1],
                 [0, 0, 1, 0]])

display(Math(r"\mathrm{CNOT} = " + sp.latex(sp.Matrix(CNOT))))
display(Math(r"\mathrm{CNOT}^\dagger\,\mathrm{CNOT} = "
             + sp.latex(sp.Matrix(CNOT.conj().T @ CNOT))))

print("Problem 8  --  unitary?",
      np.allclose(CNOT.conj().T @ CNOT, np.eye(4)),
      np.allclose(CNOT @ CNOT.conj().T, np.eye(4)))
```

$$\mathrm{CNOT} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

$$\mathrm{CNOT}^\dagger\,\mathrm{CNOT} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

```
Problem 8  --  unitary? True True
```

1.7 9. Implementation of a Quantum Circuit

[7]:
```
# --- Problem 9: NOT on the MSB, then CNOT with the MSB as the control ---
I2 = np.eye(2)
ket00 = np.kron(ket0, ket0)   # the register starts in |00>

state = CNOT @ (np.kron(X, I2) @ ket00)   # X on the MSB (first factor), then CNOT
display(Math(r"\mathrm{CNOT}\,(X \otimes I)\ket{00} = "
             + sp.latex(sp.Matrix(np.round(state, 4)).T)))

for label, amplitude in zip(["00", "01", "10", "11"], state):
    print(f"P(|{label}>) = {abs(amplitude) ** 2:.2%}")
```

$$\mathrm{CNOT}\,(X \otimes I)\,|00\rangle = \begin{bmatrix}0.0 & 0.0 & 0.0 & 1.0\end{bmatrix}$$

```
P(|00>) = 0.00%
P(|01>) = 0.00%
P(|10>) = 0.00%
P(|11>) = 100.00%
```